VIBECON · 4-MINUTE PITCH

Crystal

Clean code that ships.

The architecture guardian for vibe-coded projects.

crystalcodes.dev · open source · MIT

THE PROBLEM

AI builds fast. Then things break.

You ship something in Cursor at 11pm. It works.

Next session, you add a feature. **Old stuff breaks.**

A password lands in the frontend. A test disappears. Your handoff prompt is 4 sentences of guesswork.

- **36%** of people using AI to code never test it
- **1.7×** more bugs in AI-written vs human-written code
- **45%** of AI-generated code has security holes

*Not lint. Not CI. These are **project-level** integrity failures.*

THE INSIGHT

The bug isn't in one file. It's between sessions.

Every linter watches one file. Every CI watches one build.

Nothing watches your project across AI sessions.

- Domain boundaries erode (DB code slides into frontend)
- Secrets leak into source (API keys, hardcoded passwords)
- Tests silently disappear
- Context is lost on every new prompt

Crystal is the first tool built for this exact gap — **the handoff, not the file.**

WHO IT'S FOR

Built for vibe coders. By a vibe coder.

Persona

Anyone shipping a product with Cursor, Claude, Windsurf, Bolt, Replit, or Emergent — solo or in small teams. Not enterprise. **You.**

Why this team

I shipped this on Emergent. I hit every one of these pain points in the last 30 days. Crystal is the tool I built because I needed it first.

DEMO — LIVE TERMINAL

One install. Sixteen commands.

```
$ pip install crystal-code
  Installed crystal-code-0.3.0

$ crystal init
  Found: React + Python + MongoDB
  16 quality gates + dependency audit loaded
```

Switch to terminal. Open a real broken repo from your dashboard.

DEMO — CRYSTAL CHECK

Health: F (0/100) → A (98/100)

```
$ crystal check examples/case-study/before

[CRITICAL] sec-001  src/config.js:15  Hardcoded API key
[CRITICAL] dom-001  src/App.jsx:8     DB code in frontend
[HIGH]     sec-005  backend/server.py CORS wildcard allows all origins
[HIGH]     arch-002 .gitignore missing — secrets will hit GitHub

Health: F (0/100) · 29 issues · 5 critical · STAGE FAILED
```

In plain English. No config. No setup.

DEMO — FIX-PROMPT + AUDIT

Paste-ready AI prompts. Senior-engineer review.

`crystal fix-prompt` → generates a paste-ready prompt for your AI tool, with *why this is dangerous* in plain English.

`crystal audit --llm` → Crystal Agent correlates issues across analyzers, finds hotspots, calls Claude/GPT for senior-engineer insight. **Works with your key or the Emergent universal key.**

`crystal handoff` → one paste. Your next AI session knows the full context.

DEMO — PRODUCTION GATE

Three stages. Escalating strictness.

```
$ crystal check --stage production
  Tests required: 0 failing allowed
  No localhost URLs
  No TODOs
```

```
All dependencies audited
Stage: PRODUCTION · BLOCKED · 2 criticals

Cannot ship. Run crystal fix-prompt to resolve.
```

MCP-native: plugs straight into Cursor, Claude Desktop, Windsurf. Your AI sees the gates in real time.

EXECUTION

Shipped. Published. Tested.

- **Live on PyPI**: `pip install crystal-code` — v0.3.0, real users can install *right now*
- **161 tests, 82% coverage** — not slideware
- **16 commands, 5 analyzers, MCP server, stage pipeline**
- **crystalcodes.dev** — live landing page with full SEO, OG card, favicon

Built with Emergent

Full-stack, end-to-end. Landing page + backend API + Python CLI + test suite + MCP server. The `crystal audit --llm` feature **uses Emergent's universal LLM key** — Crystal dogfoods the same AI substrate it protects.

WHY NOW · WHAT'S NEXT

Every AI-coded project needs this. None have it.

- **TAM**: 10M+ developers now use AI coding tools (Cursor, Claude, Bolt, Replit, Emergent). Growing ~15% QoQ.
- **Path to 100 users**: PyPI + HackerNews launch, Cursor/Claude marketplace, 5 stack presets ship today.
- **Path to revenue**: team tier ($19/mo) for shared rules + PR dashboard + history across repos.

What I'm asking from Vibecon judges

feedback on the two hard trade-offs [redacted] and depth.

Crystal. Clean code that ships.

crystalcodes.dev · `pip install crystal-code` · MIT · built with Emergent

APPENDIX · TECHNICAL DEPTH

Architecture

- **5 analyzers**: architecture, domain purity, security, hygiene, dependencies
- **Stage pipeline**: `local` (lenient) → `staging` (strict) → `production` (zero tolerance)
- **16+ gates, rule-based** — no LLM required for core scoring
- **Optional LLM layer**: Anthropic / OpenAI / Emergent universal key
- **MCP server**: 8 tools, 3 resources, 2 prompts via FastMCP
- **Per-rule auto-fixers**: idempotent, whitelisted, never touches source code
- **Test suite**: 161 tests across 16 files, 82% coverage, CI-ready

---

APPENDIX · CASE STUDY

Before → After (real numbers)

Metric	Before	After
Grade	F	**A**
Score	0 / 100	**98 / 100**
Critical issues	5	**0**
High issues	3	**0**
Total issues	29	**1**

Stage gate	BLOCKED	**PASSED**

Project: React + FastAPI + MongoDB todo app. Fix time: ~2 hours with `crystal fix-prompt`.